Informe Técnico: FNBC y Desnormalización Controlada en Food Store

Cátedra: Base de Datos II · Comisión: 2PRO4 · Docente: Sergio Neira. Integrantes: Tiziano Valentini, Juan Martin Garcia.

Parte 1: Análisis y aplicación de la FNBC

1.1. Dependencias funcionales (4.2.a)

Sobre R(LoteID, DepositoID, ResponsableControlID), las reglas de logística determinan: DF1: {LoteID, DepositoID} -> ResponsableControlID, porque cada lote y depósito tiene un único responsable; DF2: ResponsableControlID -> DepositoID, porque cada responsable pertenece a un único depósito.

1.2. Clausuras, claves candidatas y atributos primos (4.2.b)

Conjunto X	Clausura X+	¿Superclave?
{LoteID}	{LoteID}	No
{DepositoID}	{DepositoID}	No
{ResponsableControlID}	{ResponsableControlID, DepositoID}	No
{DepositoID, ResponsableControlID}	{DepositoID, ResponsableControlID}	No
{LoteID, DepositoID}	R, por DF1	Sí
{LoteID, ResponsableControlID}	R, por DF2	Sí

LoteID no aparece a la derecha de ninguna DF y debe estar en toda clave. LoteID solo no alcanza; agregar DepositoID o ResponsableControlID produce una superclave mínima. Por tanto, el conjunto completo de claves candidatas es {{LoteID, DepositoID}, {LoteID, ResponsableControlID}}. Los tres atributos son primos y no hay atributos no primos.

1.3. Demostración de violación de FNBC (4.2.c)

R está en FNBC si todo determinante de una DF no trivial es superclave. DF1 cumple: {LoteID, DepositoID}+ = R. DF2 no cumple: {ResponsableControlID}+ = {ResponsableControlID, DepositoID}, que no contiene LoteID. DF2 es no trivial y su determinante no es superclave; por eso control_lote_almacen no está en FNBC. Sí cumple 3FN porque DepositoID es primo, condición alternativa que FNBC no admite.

1.4. Anomalías sobre la instancia provista (4.2.d)

Inserción. El inspector 803 se incorpora al depósito 32. No puede guardarse 803 -> 32 sin asociarlo a un lote, porque lote_id integra la PK y es NOT NULL. El hecho maestro de pertenencia queda condicionado a un control de lote.

Borrado. Si se cancela el control del lote 503 y se elimina (503,31,802), se pierde el único registro de que 802 pertenece al depósito 31. Borrar un control elimina un dato independiente de la dotación de personal.

Actualización. El depósito 30 del responsable 801 está repetido en los lotes 501 y 502. Trasladarlo al depósito 32 exige actualizar ambas filas. Si se modifica una sola, 801 queda asociado a dos depósitos y viola DF2; la PK (lote_id, deposito_id) no lo impide.

1.5. Descomposición sin pérdida (4.2.e)

Se aplica el algoritmo sobre X -> Y, con X={ResponsableControlID}, Y={DepositoID}: R1=X unión Y={ResponsableControlID, DepositoID}; R2=R-(Y-X)={LoteID, ResponsableControlID}. R1 está en FNBC porque su determinante es clave. R2 solo tiene DFs triviales proyectadas.

```
CREATE TABLE responsable_deposito (
 responsable_control_id BIGINT PRIMARY KEY REFERENCES usuario(id),
 deposito_id BIGINT NOT NULL REFERENCES deposito(id)
);
CREATE TABLE asignacion_control_lote (
 lote_id BIGINT NOT NULL REFERENCES lote(id),
 responsable_control_id BIGINT NOT NULL
  REFERENCES responsable_deposito(responsable_control_id),
 PRIMARY KEY (lote_id, responsable_control_id)
);
CREATE VIEW vista_control_lote_almacen_fnbc AS
SELECT lote_id, deposito_id, responsable_control_id
FROM asignacion_control_lote NATURAL JOIN responsable_deposito;
```

1.6. Reunión sin pérdida y migración (4.2.f)

El atributo común es ResponsableControlID. Su clausura contiene todos los atributos de R1 y es superclave de esa relación. El criterio de superclave sobre la intersección demuestra que la reunión natural es sin pérdida respecto de F. El script migra con INSERT ... SELECT DISTINCT y verifica EXCEPT en ambos sentidos.

```
Vista: (501,30,801), (502,30,801), (503,31,802)
Original EXCEPT vista: 0 filas; vista EXCEPT original: 0 filas
Cardinalidades: original = 3; vista = 3
```

La descomposición no preserva DF1 mediante PK/FK. El script valida la reunión al modificar asignaciones o responsables, con un bloqueo común de escrituras en READ COMMITTED. Las pruebas rechazan un segundo responsable del mismo depósito para un lote y un traslado incompatible. También demuestra las tres anomalías originales y su resolución en las tablas descompuestas, sin alterar la instancia de ejemplo.

Parte 2: Desnormalización controlada

2.1. Consulta de partida y adaptación al esquema real

La consulta del punto 5.1 se adapta a los nombres del esquema de Food Store sin cambiar su significado:

Consigna	Esquema real de Food Store
ped.fecha = CURRENT_DATE	fecha_hora TIMESTAMPTZ en [CURRENT_DATE, CURRENT_DATE+1)
dp.subtotal	dp.cantidad * dp.precio_unitario
producto_id / pedido_id	id_producto / id_pedido
pr.id / c.id / ped.id	id_producto / id_categoria / id_pedido
c.nombre	nombre_categoria
dp.eliminado / ped.eliminado	Columnas incorporadas por soft_delete.sql

```
SELECT c.nombre_categoria AS categoria,
       SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
FROM Detalle_Pedido dp
JOIN Producto pr ON pr.id_producto = dp.id_producto
JOIN Categoria c ON c.id_categoria = pr.id_categoria
JOIN Pedido ped ON ped.id_pedido = dp.id_pedido
WHERE ped.fecha_hora >= CURRENT_DATE
  AND ped.fecha_hora < CURRENT_DATE + 1
  AND dp.eliminado = FALSE AND ped.eliminado = FALSE
GROUP BY c.nombre_categoria
ORDER BY total_vendido DESC LIMIT 5;
```

Instancia de desarrollo poblada con data.sql del proyecto: 20.003 clientes, 12 categorías, 50.005 productos, 200.003 pedidos y 400.011 detalles. La preparación del día deja 10.198 pedidos y 20.396 detalles vigentes el 13/10/2026. Motor: PostgreSQL 16.15, Windows de 64 bits. Zona: America/Argentina/Buenos_Aires. shared_buffers=128MB, work_mem=4MB y max_parallel_workers_per_gather=2. Se conservan las restricciones e índices de TP2-TP5.

2.2. Medición antes y diagnóstico (5.2.a)

EXPLAIN (ANALYZE, BUFFERS) de la consulta normalizada dio Execution Time=77,726 ms en el plan completo del anexo A. El nodo de lectura dominante es Parallel Seq Scan sobre detalle_pedido: recorre 400.011 filas en total, con 133.337 por participante y tres loops. Parallel Hash Join filtra por pedidos del día; dos Hash Join adicionales incorporan Producto y Categoria. El catálogo de productos requiere leer 50.005 filas. Los tiempos de los nodos padres incluyen el trabajo de sus hijos; no se suman entre sí ni entre workers como tiempo total.

2.3. Patrón elegido y justificación (5.2.b)

Se elige columna precalculada con disparadores: Detalle_Pedido.categoria_nombre. El plan normalizado de 77,726 ms muestra el recorrido del detalle y las reuniones con el catálogo; almacenar el nombre evita las reuniones con Producto y Categoria y la lectura medida baja a 66,880 ms. Un disparador recalcula el nombre al insertar o modificar producto/valor redundante de un detalle; otros propagan renombres de categoría y reclasificaciones de producto en la misma transacción, con un bloqueo común antes de escribir las fuentes para evitar carreras en READ COMMITTED. Una auditoría compara cada nombre contra su fuente. Es reversible sin pérdida: eliminar la columna y sus disparadores conserva todos los datos fuente. El costo es espacio adicional, propagaciones masivas ante cambios de catálogo y menor concurrencia de escrituras; el reporte refleja la categoría vigente, igual que la consulta original.

2.4. Implementación (5.2.c)

El script tp_desnormalizacion_top_categorias.sql agrega categoria_nombre VARCHAR(80), carga los valores desde Producto JOIN Categoria y exige NOT NULL. La instalación es atómica. Los disparadores cubren: alta o cambio de producto/nombre en un detalle, renombre de categoría y cambio de categoría de un producto. Se conservan los disparadores de stock e inmutabilidad del detalle del proyecto. La fecha y el monto no se duplican.

2.5. Consulta nueva y comparación (5.2.d)

```
SELECT dp.categoria_nombre AS categoria,
       SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
FROM Detalle_Pedido dp
JOIN Pedido ped ON ped.id_pedido = dp.id_pedido
WHERE ped.fecha_hora >= CURRENT_DATE
  AND ped.fecha_hora < CURRENT_DATE + 1
  AND dp.eliminado = FALSE AND ped.eliminado = FALSE
GROUP BY dp.categoria_nombre
ORDER BY total_vendido DESC LIMIT 5;
```

Antes (4 tablas)	Después (columna precalculada)
Execution Time: 77,726 ms	Execution Time: 66,880 ms
Lectura dominante: Parallel Seq Scan de detalle; Parallel Hash Join y dos Hash Join de catálogo	Lectura dominante: Parallel Seq Scan de detalle y Parallel Hash Join con Pedido
Mediana de 7 repeticiones: 77,258 ms	Mediana de 7 repeticiones: 65,942 ms

La consulta nueva lee el dato redundante y conserva Pedido para los filtros de fecha y anulación. EXCEPT ALL en ambos sentidos sobre todos los grupos del día dio 0 diferencias. En siete pares con orden alternado y el mismo estado físico e índices, la mediana disminuyó 14,65%. Las capturas de los anexos son corridas adicionales; sus tiempos no se confunden con las medianas. Las repeticiones completas están en evidencias/planes_mediciones.json.

2.6. Auditoría de sincronización (5.2.e)

```
SELECT dp.id_pedido, dp.id_producto,
       dp.categoria_nombre AS valor_desnormalizado,
       c.nombre_categoria AS valor_fuente_verdad
FROM Detalle_Pedido dp
JOIN Producto pr ON pr.id_producto = dp.id_producto
JOIN Categoria c ON c.id_categoria = pr.id_categoria
WHERE dp.categoria_nombre IS DISTINCT FROM c.nombre_categoria;
```

```
 id_pedido | id_producto | valor_desnormalizado | valor_fuente_verdad
-----------+-------------+----------------------+--------------------
(0 rows)
```

El script incorpora tres pruebas con ROLLBACK, como en el diseño original: P1 inserta un detalle con un nombre falso y verifica que se derive el real; P2 renombra una categoría; P3 cambia la categoría de un producto. Después de P2 y P3, la auditoría vuelve a dar 0 filas. Las salidas completas están en evidencias/salida_parte2.txt.

```
P1: nombre ingresado = VALOR FALSO; nombre guardado = Pizzas
P2: detalles con nombre nuevo = 33715; desincronizados = 0
P3: producto 1 pasa a categoría 2; nombre = Bebidas;
    desincronizados = 0
```

Las pruebas auxiliares cubren además edición directa de la columna, cambio de producto, nombres de categoría repetidos, bajas lógicas, cambio de día y borrado en cascada: auditoría=0 y diferencias=0 en cada etapa. En dos transacciones concurrentes, la inserción esperó 4,022 s por el cambio de catálogo y derivó el nombre confirmado; auditoría global=0. Se comprobó también la reversión sin cambios en las filas y suma monetaria fuente.

2.7. Alcance y límites

La comparación conserva los índices del proyecto. El Parallel Seq Scan de detalle sigue presente: la columna elimina reuniones, pero no precalcula los totales del día. Un renombre puede actualizar muchas filas. El bloqueo común permite SELECT y serializa escrituras sobre las fuentes; requiere READ COMMITTED y los disparadores habilitados. No se midió rendimiento de escrituras.

Las siete repeticiones incluyeron dos tiempos altos en la consulta nueva (147,232 y 149,024 ms), conservados en las evidencias. Hubo shared read incluso después del calentamiento. La reducción de la mediana describe esta instancia; no garantiza la misma latencia en otras condiciones. Si se necesitara la categoría histórica de cada venta, cambiaría la semántica respecto de la consulta original.

Los planes completos de EXPLAIN ANALYZE se incluyen en los anexos A y B. Los scripts SQL y el informe se complementan: las implementaciones y verificaciones son ejecutables y las evidencias registran sus resultados.

Anexo A: EXPLAIN ANALYZE antes

Captura de la salida completa de psql; se conservan todos los nodos y métricas.

```
psql | EXPLAIN (ANALYZE, BUFFERS) | antes
```

```
QUERY PLAN
 Limit  (cost=17317.00..17317.01 rows=5 width=44) (actual time=71.667..77.357 rows=5 loops=1)
   Buffers: shared hit=9424 read=2916
   ->  Sort  (cost=17317.00..17317.03 rows=12 width=44) (actual time=71.665..77.354 rows=5 loops=1)
         Sort Key: (sum(((dp.cantidad)::numeric * dp.precio_unitario))) DESC
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=9424 read=2916
         ->  Finalize GroupAggregate  (cost=17313.67..17316.80 rows=12 width=44) (actual time=71.627..77.333 rows=12 loops=1)
               Group Key: c.nombre_categoria
               Buffers: shared hit=9421 read=2916
               ->  Gather Merge  (cost=17313.67..17316.47 rows=24 width=44) (actual time=71.619..77.314 rows=36 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=9421 read=2916
                     ->  Sort  (cost=16313.64..16313.67 rows=12 width=44) (actual time=47.306..47.309 rows=12 loops=3)
                           Sort Key: c.nombre_categoria
                           Sort Method: quicksort  Memory: 26kB
                           Buffers: shared hit=9421 read=2916
                           Worker 0:  Sort Method: quicksort  Memory: 26kB
                           Worker 1:  Sort Method: quicksort  Memory: 26kB
                           ->  Partial HashAggregate  (cost=16313.28..16313.43 rows=12 width=44) (actual time=47.269..47.274 rows=12 loops=3)
                                 Group Key: c.nombre_categoria
                                 Batches: 1  Memory Usage: 24kB
                                 Buffers: shared hit=9405 read=2916
                                 Worker 0:  Batches: 1  Memory Usage: 24kB
                                 Worker 1:  Batches: 1  Memory Usage: 24kB
                                 ->  Hash Join  (cost=3674.61..16229.56 rows=8372 width=22) (actual time=14.777..45.096 rows=6799 loops=3)
                                       Hash Cond: (pr.id_categoria = c.id_categoria)
                                       Buffers: shared hit=9405 read=2916
                                       ->  Hash Join  (cost=3673.34..16198.55 rows=8372 width=18) (actual time=14.380..43.833 rows=6799 loops=3)
                                             Hash Cond: (dp.id_producto = pr.id_producto)
                                             Buffers: shared hit=9372 read=2916
                                             ->  Parallel Hash Join  (cost=2032.22..14535.46 rows=8372 width=18) (actual time=3.924..31.529 rows=6799 loops=3)
                                                   Hash Cond: (dp.id_pedido = ped.id_pedido)
                                                   Buffers: shared hit=7824 read=2916
                                                   ->  Parallel Seq Scan on detalle_pedido dp  (cost=0.00..12065.71 rows=166671 width=26) (actual time=2.414..20.695 rows=133337 loops=3)
                                                         Filter: (NOT eliminado)
                                                         Buffers: shared hit=7483 read=2916
                                                   ->  Parallel Hash  (cost=1958.36..1958.36 rows=5909 width=8) (actual time=1.442..1.442 rows=3399 loops=3)
                                                         Buckets: 16384  Batches: 1  Memory Usage: 544kB
                                                         Buffers: shared hit=341
                                                         ->  Parallel Bitmap Heap Scan on pedido ped  (cost=271.40..1958.36 rows=5909 width=8) (actual time=1.340..3.172 rows=10198 loops=1)
                                                               Recheck Cond: ((fecha_hora >= CURRENT_DATE) AND (fecha_hora < (CURRENT_DATE + 1)))
                                                               Filter: (NOT eliminado)
                                                               Heap Blocks: exact=280
                                                               Buffers: shared hit=341
                                                               ->  Bitmap Index Scan on idx_pedido_fecha_cliente  (cost=0.00..268.89 rows=10046 width=0) (actual time=1.303..1.303 rows=10211 loops=1)
                                                                     Index Cond: ((fecha_hora >= CURRENT_DATE) AND (fecha_hora < (CURRENT_DATE + 1)))
                                                                     Buffers: shared hit=61
                                             ->  Hash  (cost=1016.05..1016.05 rows=50005 width=16) (actual time=10.354..10.354 rows=50005 loops=3)
                                                   Buckets: 65536  Batches: 1  Memory Usage: 2856kB
                                                   Buffers: shared hit=1548
                                                   ->  Seq Scan on producto pr  (cost=0.00..1016.05 rows=50005 width=16) (actual time=0.282..5.676 rows=50005 loops=3)
                                                         Buffers: shared hit=1548
                                       ->  Hash  (cost=1.12..1.12 rows=12 width=20) (actual time=0.368..0.369 rows=12 loops=3)
                                             Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                             Buffers: shared hit=3
                                             ->  Seq Scan on categoria c  (cost=0.00..1.12 rows=12 width=20) (actual time=0.357..0.358 rows=12 loops=3)
                                                   Buffers: shared hit=3
 Planning:
   Buffers: shared hit=480
 Planning Time: 8.374 ms
 Execution Time: 77.726 ms
(62 rows)
```

Anexo B: EXPLAIN ANALYZE después

Captura de la salida completa de psql; se conservan todos los nodos y métricas.

psql | EXPLAIN (ANALYZE, BUFFERS) | despues

```
QUERY PLAN
 Limit  (cost=15622.90..15622.91 rows=5 width=44) (actual time=61.405..66.748 rows=5 loops=1)
   Buffers: shared hit=7905 read=2884
   ->  Sort  (cost=15622.90..15622.93 rows=12 width=44) (actual time=61.403..66.745 rows=5 loops=1)
         Sort Key: (sum(((dp.cantidad)::numeric * dp.precio_unitario))) DESC
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=7905 read=2884
         ->  Finalize GroupAggregate  (cost=15619.57..15622.70 rows=12 width=44) (actual time=61.363..66.720 rows=12 loops=1)
               Group Key: dp.categoria_nombre
               Buffers: shared hit=7902 read=2884
               ->  Gather Merge  (cost=15619.57..15622.37 rows=24 width=44) (actual time=61.356..66.701 rows=36 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=7902 read=2884
                     ->  Sort  (cost=14619.54..14619.57 rows=12 width=44) (actual time=37.077..37.080 rows=12 loops=3)
                           Sort Key: dp.categoria_nombre
                           Sort Method: quicksort  Memory: 26kB
                           Buffers: shared hit=7902 read=2884
                           Worker 0:  Sort Method: quicksort  Memory: 26kB
                           Worker 1:  Sort Method: quicksort  Memory: 26kB
                           ->  Partial HashAggregate  (cost=14619.18..14619.33 rows=12 width=44) (actual time=37.050..37.055 rows=12 loops=3)
                                 Group Key: dp.categoria_nombre
                                 Batches: 1  Memory Usage: 24kB
                                 Buffers: shared hit=7886 read=2884
                                 Worker 0:  Batches: 1  Memory Usage: 24kB
                                 Worker 1:  Batches: 1  Memory Usage: 24kB
                                 ->  Parallel Hash Join  (cost=2032.22..14535.46 rows=8372 width=22) (actual time=3.750..34.889 rows=6799 loops=3)
                                       Hash Cond: (dp.id_pedido = ped.id_pedido)
                                       Buffers: shared hit=7886 read=2884
                                       ->  Parallel Seq Scan on detalle_pedido dp  (cost=0.00..12065.71 rows=166671 width=30) (actual time=2.507..23.403 rows=133337 loops=3)
                                             Filter: (NOT eliminado)
                                             Buffers: shared hit=7515 read=2884
                                       ->  Parallel Hash  (cost=1958.36..1958.36 rows=5909 width=8) (actual time=1.170..1.171 rows=3399 loops=3)
                                             Buckets: 16384  Batches: 1  Memory Usage: 544kB
                                             Buffers: shared hit=341
                                             ->  Parallel Bitmap Heap Scan on pedido ped  (cost=271.40..1958.36 rows=5909 width=8) (actual time=0.947..2.461 rows=10198 loops=1)
                                                   Recheck Cond: ((fecha_hora >= CURRENT_DATE) AND (fecha_hora < (CURRENT_DATE + 1)))
                                                   Filter: (NOT eliminado)
                                                   Heap Blocks: exact=280
                                                   Buffers: shared hit=341
                                                   ->  Bitmap Index Scan on idx_pedido_fecha_cliente  (cost=0.00..268.89 rows=10046 width=0) (actual time=0.921..0.921 rows=10211 loops=1)
                                                         Index Cond: ((fecha_hora >= CURRENT_DATE) AND (fecha_hora < (CURRENT_DATE + 1)))
                                                         Buffers: shared hit=61
 Planning:
   Buffers: shared hit=380
 Planning Time: 4.998 ms
 Execution Time: 66.880 ms
(46 rows)
```